Wrapper Classes, Autoboxing and the Integer Cache

9 min read Updated 28 Jul 2026 Topic 15 of 65

Why it matters

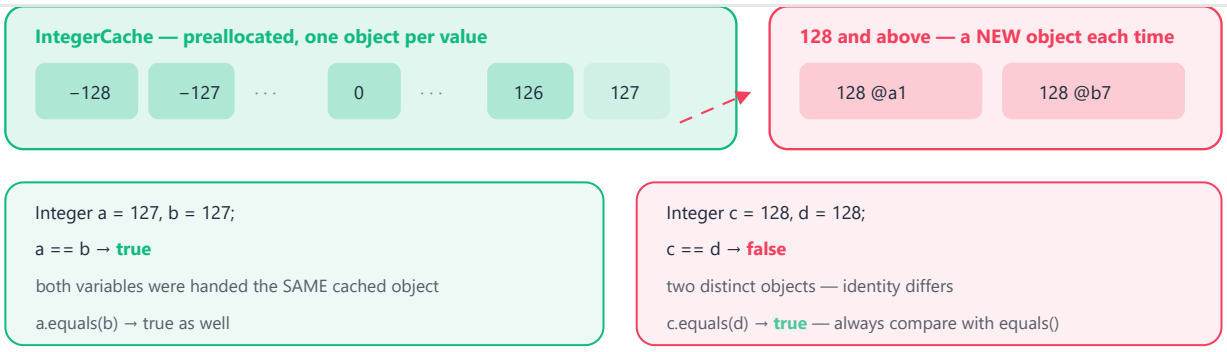
The `Integer` cache produces the most notorious "what does this print?" question in Java: the same comparison is `true` at 127 and `false` at 128. It looks like a language defect, has a precise explanation, and reveals whether a candidate understands the difference between identity and equality. Autoboxing also causes two entirely real production problems — `NullPointerException` from an innocuous-looking assignment, and allocation storms in numeric loops.

The concept, simply

Each primitive has a corresponding **wrapper class** — `int` / `Integer`, `double` / `Double`, `boolean` / `Boolean`, and so on. Wrappers exist because generics and collections work only with reference types: you cannot write `List<int>`.

Autoboxing is the compiler automatically converting between the two. `Integer x = 5;` becomes `Integer.valueOf(5)`; `int y = x;` becomes `x.intValue()`. Both are silent, and both matter.

The cache is the twist. `Integer.valueOf` does not allocate for small values — it returns a shared object from a preallocated table:



Boxing calls Integer.valueOf, which returns a cached shared instance for -128…127 and a brand-new object above that. Identity changes; equality never does.

How it works

The **cached ranges** are specified, not incidental. The JLS requires boxing to return identical objects for: `Boolean` `true`/`false`, all `Byte` values, `Character` 0–127, and `Short`, `Integer` and `Long` from -128 to 127. Above that range, behaviour is implementation-defined, and HotSpot allocates a new object. `Float` and `Double` are never cached at all.

The `Integer` upper bound is tunable with `-XX:AutoBoxCacheMax=<n>`, which is a delightful interview fact and a terrible thing to depend on: it means `==` behaviour on boxed integers can differ between two JVMs running the same code.

Mixed-type comparison is the rule that makes it all navigable: if either operand of `==` is a primitive, the other is unboxed and the values are compared numerically. Only when *both* sides are wrappers does `==` compare references.

Code examples

The famous puzzle:

JAVA

Copy

```java
Integer a = 127, b = 127;
Integer c = 128, d = 128;

System.out.println(a == b);        // true   - same cached object
```

```
int e = 128;
System.out.println(c == e);        // true   - e is a primitive, so c is
                                   //          UNBOXED and 128 == 128
```

The production NPE, which is the reason this topic is more than trivia:

JAVA Copy

```
Map<String, Integer> counts = new HashMap<>();
int n = counts.get("missing");     // ✗ NullPointerException
// get() returns null; assigning to int inserts null.intValue().
// The line looks like a plain assignment - nothing hints at a method call

// Fixes, in order of preference:
int n = counts.getOrDefault("missing", 0);        // ✓ clearest
Integer boxed = counts.get("missing");            // ✓ keep it nullable
int n = boxed != null ? boxed : 0;

// The same trap in a ternary - this throws when the map misses, because t
// two branches have types Integer and int, forcing an unboxing of null:
int value = flag ? counts.get(key) : 0;
```

The performance trap:

JAVA Copy

```
// ✗ Allocates ~2 million Integer objects. Long is worse - every += boxes.
Long sum = 0L;
for (long i = 0; i < 1_000_000; i++) {
    sum += i;                      // unbox, add, box - every iteration
}

// ✓ No allocation at all.
long sum = 0L;
for (long i = 0; i < 1_000_000; i++) sum += i;

// Same reasoning in streams - the primitive specialisations exist for thi
```

And a nastier one, because it is silently wrong rather than slow:

```java
List<Integer> list = new ArrayList<>(List.of(10, 20, 30));
list.remove(1);                    // removes the ELEMENT AT INDEX 1 → 20
list.remove(Integer.valueOf(10));  // removes the OBJECT 10

// remove(int) and remove(Object) are overloads. An int argument picks the
// index version — Chapter 2's overload resolution, biting in the standard
// library. Boxing explicitly is the only way to say what you mean.
```

Under the hood

Why cache at all? Small integers are overwhelmingly the common case — loop counters, sizes, status codes, map values. Preallocating 256 objects at class-initialisation time removes an enormous number of allocations from typical programs. The cache is a genuine optimisation; the `==` weirdness is a side effect of it, not its purpose.

The memory cost of wrappers justifies the whole primitive/wrapper split from Chapter 1. An `Integer` is a heap object: roughly 16 bytes with its header, plus a reference of 4 or 8 bytes to reach it — about 20 bytes to hold a value that fits in 4. A `List<Integer>` of a million elements costs on the order of 20 MB against 4 MB for an `int[]`, spread across the heap so that iteration misses cache constantly, and every element adds GC work. This is why `IntStream`, `LongStream` and `DoubleStream` exist as separate types, and why Project Valhalla is trying to erase the distinction.

One more genuinely useful modern note: `Integer` and friends carry static helpers that are the right way to do several common things — `Integer.parseInt` (returns a primitive) versus `Integer.valueOf` (returns a boxed, possibly cached instance), `Integer.compare(a, b)` for safe comparison without overflow, and `Integer.MAX_VALUE` / `MIN_VALUE`. Since Java 9 the `new Integer(5)` constructor is deprecated, and removed in later releases — it always allocated, which is exactly what you do not want.

- **Comparing wrappers with `==`.** Works below 128 and fails above it, so it survives testing and breaks in production.
- **Unboxing a possibly-null wrapper.** The most common NPE in enterprise Java. Use `getOrDefault` or keep the variable boxed.
- **Boxing in hot loops.** Declare the accumulator as a primitive; use `IntStream` / `LongStream`.
- `list.remove(someInt)`. Picks `remove(int index)`. Wrap with `Integer.valueOf( ... )` when you mean the value.
- **Using `new Integer( ... )`.** Deprecated and removed; always allocates. Use `Integer.valueOf` or just autobox.
- **Depending on the cache range.** It is tunable with `-XX:AutoBoxCacheMax`, so identical code can behave differently on two JVMs.
- **Assuming `Double` is cached.** Floating-point wrappers never are — `Double.valueOf(1.0) == Double.valueOf(1.0)` is false.

Interview lens

Q: "Why is `Integer a = 127, b = 127; a == b` true but false at 128?"
A: Autoboxing calls `Integer.valueOf`, which returns a shared cached object for -128 to 127 and allocates a new one above that. So at 127 both variables reference the same object and `==` is true; at 128 they are distinct objects. `equals` is true in both cases, which is why it is the only correct way to compare wrappers.

Q: "Which values are cached?"
A: The JLS mandates caching for `Boolean`, all `Byte` values, `Character` 0–127, and `Short` / `Integer` / `Long` from -128 to 127. `Float` and `Double` are never cached. The `Integer` upper bound can be raised with `-XX:AutoBoxCacheMax`, which is exactly why relying on `==` is unsafe — the same code can behave differently on two JVMs.

A: The compiler inserting conversions between primitives and wrappers — `Integer.valueOf` one way, `intValue()` the other. Two costs. Unboxing a null throws `NullPointerException` at a line that looks like a plain assignment. And boxing in a loop allocates an object per iteration, which is silent GC pressure — declare accumulators as primitives and use the primitive stream types.

Q: "`Integer x = null; int y = x;` — what happens?"

A: `NullPointerException`, because the assignment compiles to `x.intValue()` and `x` is null. It is worth stressing that no cast or method call is visible in the source, which is why this bug hides so well in code that reads database results or map lookups.

Q: "Why do wrapper classes exist?"

A: Generics and collections only work with reference types, so there is no `List<int>`. Wrappers also give primitives a place to hang useful statics — `parseInt`, `MAX_VALUE`, `compare` — and allow null to represent "absent", which a primitive cannot.

Q: "`list.remove(1)` on a `List<Integer>` — what is removed?"

A: The element at index 1, not the value 1. `remove(int)` and `remove(Object)` are overloads, and overload resolution happens at compile time on the declared argument type, so an `int` literal selects the index version. To remove by value you must box explicitly: `list.remove(Integer.valueOf(1))`.

Q: "`parseInt` or `valueOf`?"

A: `Integer.parseInt` returns a primitive `int`; `Integer.valueOf` returns an `Integer`, using the cache where it applies. Use `parseInt` when you want a

Cheat-card

- Boxing calls `Integer.valueOf` → **cached −128…127**, new object above.
- Cached: `Boolean`, all `Byte`, `Character` 0–127, `Short` / `Integer` / `Long` −128…127. **Never** `Float` / `Double`.
- `==` on two wrappers = identity. **Always** `equals`.
- Wrapper vs primitive in `==` → the wrapper is **unboxed**, values compared.
- Unboxing null → **NPE**. `getOrDefault` is the fix.
- Boxing in loops → allocation storm. Use primitives and `IntStream` / `LongStream`.
- `list.remove(1)` = **by index**; `list.remove(Integer.valueOf(1))` = by value.
- `new Integer(...)` is **deprecated and removed**.
- Cache max is tunable (`-XX:AutoBoxCacheMax`) → never depend on `==`.

QUICK CHECK — no marks, just for you.

1. Integer a = 127, b = 127, c = 128, d = 128. What are a == b and c == d?

- ◯ true and true
- ◯ false and false
- ◯ false and true
- ◯ true and false — valueOf returns a cached shared object for -128 to 127 and a new object above that

Check answer

2. Which wrapper types does the JLS guarantee are cached?

Boolean, all Byte values, Character 0–127, and Short/Integer/Long from -128 to 127 — Float and Double are never cached

- ◯ Only Integer, from -128 to 127
- ◯ Only Boolean and Character
- ◯ All of them, across their full ranges

Check answer

3. List<Integer> list contains 10, 20, 30. What does list.remove(1) do?

- ◯ Removes the value 1, leaving the list unchanged since it is not present
- ◯ Removes the element at index 1, which is 20 — remove(int) and remove(Object) are overloads and an int literal selects the index version
- ◯ Removes both the element at index 1 and any element equal to 1
- ◯ Throws, because the argument is ambiguous

Check answer

‹ Previous
String vs StringBuilder vs StringBuffer

Next ›
The Collection Hierarchy

LEARN MORE

ARTICLES & DOCS

Boxing conversion (JLS §5.1.7)
docs.oracle.com

VIDEOS

"wrappers and integer cache" — video explainers
YouTube

wrappers and integer cache interview questions
YouTube

Hand-picked references for this lesson.